

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода

Общее задание

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).

2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.

3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.

4. Выполните пункты 1-3 для датасетов из ЛР 2.

5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№	Выборка	Тип задачи	Целевая переменная
17	https://archive.ics.uci.edu/dataset/503/hepatitis+c+virus+hcv+for+egyptian+patients	классификация	Baselinehistological staging

Ход работы

Код программы:

```
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from ucimlrepo import fetch_ucirepo

HIDDEN = [64, 32, 16]
AE_EPOCHS = 30
FT_EPOCHS = 60
BATCH = 64
LR_AE = 1e-3
LR_FT = 1e-3
SEEDS = [0, 1, 2, 3, 4]
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def load_hcv():
    ds = fetch_ucirepo(id=503)
    X = ds.data.features.copy()
    y = ds.data.targets.iloc[:, 0]
    X = X.apply(pd.to_numeric, errors="coerce").fillna(X.median(numeric_only=True))
    return X.values.astype(np.float32), y.values, "numeric"

def load_mushroom():
    ds = fetch_ucirepo(id=73)
    df = pd.concat([ds.data.features, ds.data.targets], axis=1)
    df = df.replace("?", np.nan).dropna()
    X = df.drop(columns=["poisonous"])
    return X, y, "categorical"

def prepare(X, y, kind, seed):
```

```

y = LabelEncoder().fit_transform(y)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=seed)
if kind == "numeric":
    sc = StandardScaler().fit(X_tr)
    X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)
else:
    enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(X_tr)
    X_tr, X_te = enc.transform(X_tr), enc.transform(X_te)
return (X_tr.astype(np.float32), X_te.astype(np.float32),
        y_tr.astype(np.int64), y_te.astype(np.int64))

def build_mlp(in_dim, n_classes):
    layers, prev = [], in_dim
    for h in HIDDEN:
        layers += [nn.Linear(prev, h), nn.ReLU()]
        prev = h
    layers.append(nn.Linear(prev, n_classes))
    return nn.Sequential(*layers)

def train_classifier(model, X_tr, y_tr, X_te, y_te, seed):
    torch.manual_seed(seed)
    model.to(device)
    opt = optim.Adam(model.parameters(), lr=LR_FT)
    crit = nn.CrossEntropyLoss()
    loader = DataLoader(TensorDataset(torch.tensor(X_tr), torch.tensor(y_tr)),
                        batch_size=BATCH, shuffle=True)
    Xte = torch.tensor(X_te).to(device)
    yte = torch.tensor(y_te).to(device)
    hist = {"train_loss": [], "test_loss": []}
    for _ in range(FT_EPOCHS):
        model.train() tot = 0.0
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            loss = crit(model(xb), yb)
            loss.backward()
            opt.step()
            tot += loss.item() * len(xb)
        hist["train_loss"].append(tot / len(X_tr))
        model.eval()
        with torch.no_grad():
            hist["test_loss"].append(crit(model(Xte), yte).item())
    model.eval()
    with torch.no_grad():
        pred = model(Xte).argmax(1).cpu().numpy()
    return pred, hist

def pretrain_layers(model, X_tr, seed):
    torch.manual_seed(seed)
    linear_layers = [m for m in model if isinstance(m, nn.Linear)][:-1] # без выходного слоя
    H = torch.tensor(X_tr).to(device)
    ae_losses = []
    for k, layer in enumerate(linear_layers):
        in_d, out_d = layer.in_features, layer.out_features
        enc = nn.Sequential(nn.Linear(in_d, out_d), nn.ReLU()).to(device)
        dec = nn.Linear(out_d, in_d).to(device)
        opt = optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=LR_AE)
        crit = nn.MSELoss()
        loader = DataLoader(TensorDataset(H), batch_size=BATCH, shuffle=True)
        losses = []
        for _ in range(AE_EPOCHS):
            tot = 0.0
            for (xb,) in loader:
                opt.zero_grad()
                loss = crit(dec(enc(xb)), xb)
                loss.backward()
                opt.step()

```

```

        tot += loss.item() * len(xb)
        losses.append(tot / len(H))
    ae_losses.append(losses)
    layer.weight.data.copy_(enc[0].weight.data)
    layer.bias.data.copy_(enc[0].bias.data)
    with torch.no_grad():
        H = enc(H)
    return ae_losses
def run_dataset(name, X, y, kind):
    print(f"\n{'=' * 20} {name} {'=' * 20}")
    rows, last = [], {}
    for seed in SEEDS:
        X_tr, X_te, y_tr, y_te = prepare(X, y, kind, seed)
        n_classes = len(np.unique(y_tr))
        for mode in ["без предобучения", "с предобучением"]:
            torch.manual_seed(seed)
            model = build_mlp(X_tr.shape[1], n_classes)
            ae_losses = None
            if mode == "с предобучением":
                ae_losses = pretrain_layers(model, X_tr, seed)
            pred, hist = train_classifier(model, X_tr, y_tr, X_te, y_te, seed)
            rows.append({
                "seed": seed, "mode": mode,
                "accuracy": accuracy_score(y_te, pred),
                "f1_macro": f1_score(y_te, pred, average="macro"),
                "f1_weighted": f1_score(y_te, pred, average="weighted"),
            })
            last[mode] = dict(y_te=y_te, pred=pred, hist=hist, ae=ae_losses)

    res = pd.DataFrame(rows)
    summary = res.groupby("mode")[["accuracy", "f1_macro", "f1_weighted"]].agg(["mean", "std"])
    print(summary.round(4).to_string())
    res.to_csv(f"results_{name}.csv", index=False)

    fig, ax = plt.subplots(2, 2, figsize=(11, 9))
    for i, mode in enumerate(["без предобучения", "с предобучением"]):
        d = last[mode]
        cm = confusion_matrix(d["y_te"], d["pred"])
        ax[0, i].imshow(cm, cmap="Blues")
        ax[0, i].set_title(f"{name}: {mode}")
        ax[0, i].set_xlabel("Предсказано")
        ax[0, i].set_ylabel("Истина")
        for a in range(cm.shape[0]):
            for b in range(cm.shape[1]):
                ax[0, i].text(b, a, cm[a, b], ha="center", va="center")
        ax[1, i].plot(d["hist"]["train_loss"], label="train")
        ax[1, i].plot(d["hist"]["test_loss"], label="test")
        ax[1, i].set_title(f"Loss ({mode})")
        ax[1, i].set_xlabel("Эпоха")
        ax[1, i].legend()
        ax[1, i].grid(True)
    plt.tight_layout()
    plt.savefig(f"{name}_results.png", dpi=150)
    plt.close()

    print("\nОтчёт по классам (последний seed, с предобучением):")
    print(classification_report(last["с предобучением"]["y_te"],
                                last["с предобучением"]["pred"], zero_division=0))

    plt.figure(figsize=(8, 5))
    for k, l in enumerate(last["с предобучением"]["ae"]):
        plt.plot(l, label=f"Автоэнкодер слоя {k + 1}")
    plt.xlabel("Эпоха")
    plt.ylabel("MSE")
    plt.title(f"{name}: предобучение слоёв")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(f"{name}_ae_pretraining.png", dpi=150)

```

```
plt.close()

if __name__ == "__main__":
    X, y, kind = load_hcv()
    run_dataset("HCV", X, y, kind)
    X, y, kind = load_mushroom()
    run_dataset("Mushroom", X, y, kind)
```

Вывод программы:

```
===== HCV =====
              accuracy      f1_macro      f1_weighted
              mean      std      mean      std      mean      std
mode
без предобучения  0.2538  0.0208  0.2514  0.0208  0.2526  0.0208
с предобучением  0.2577  0.0137  0.2548  0.0132  0.2560  0.0138

Отчёт по классам (последний seed, с предобучением):
              precision      recall      f1-score      support
0              0.23          0.20          0.21          101
1              0.24          0.21          0.22          100
2              0.27          0.36          0.31          106
3              0.27          0.25          0.26          109

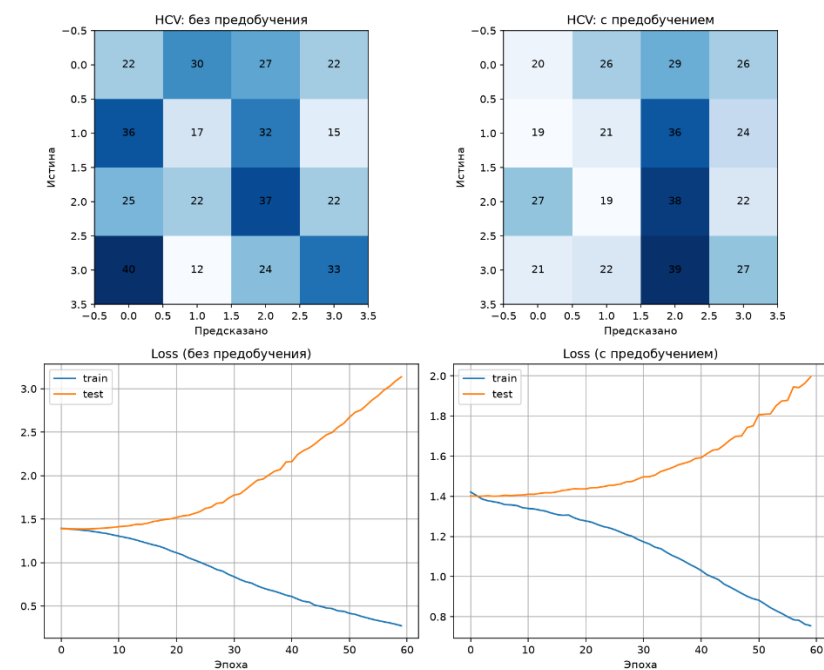
              accuracy
macro avg      0.25          0.25          0.25          416
weighted avg   0.25          0.25          0.25          416

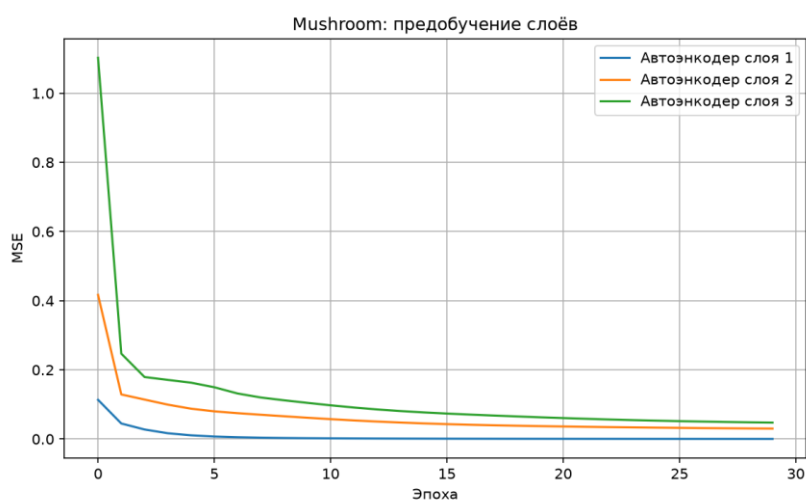
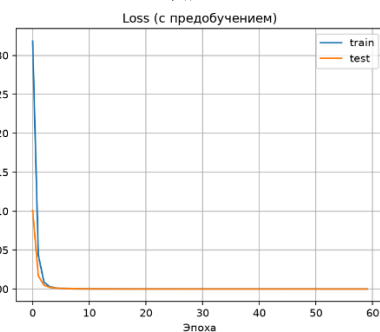
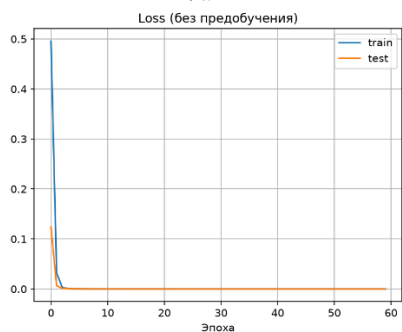
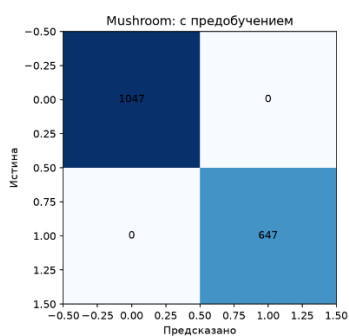
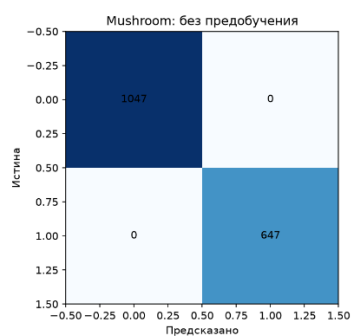
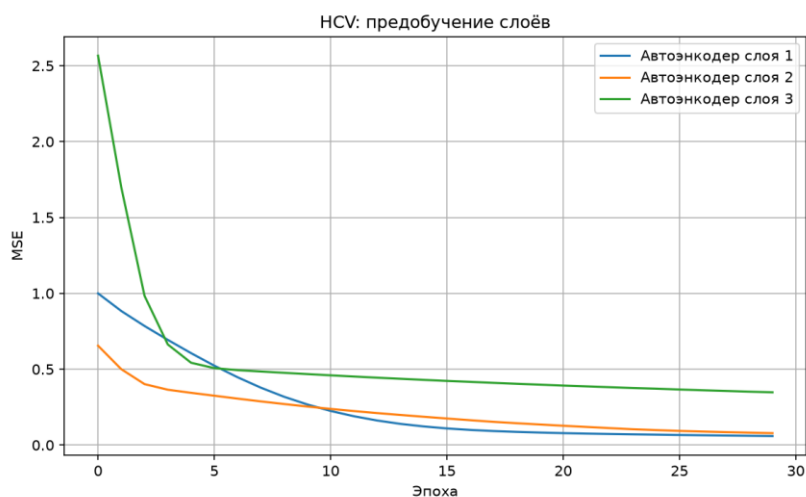
===== Mushroom =====
              accuracy      f1_macro      f1_weighted
              mean      std      mean      std      mean      std
mode
без предобучения  1.0  0.0      1.0  0.0      1.0  0.0
с предобучением  1.0  0.0      1.0  0.0      1.0  0.0

Отчёт по классам (последний seed, с предобучением):
              precision      recall      f1-score      support
0              1.00          1.00          1.00          1047
1              1.00          1.00          1.00          647

              accuracy
macro avg      1.00          1.00          1.00          1694
weighted avg   1.00          1.00          1.00          1694
```

Графики:





В работе сравнивалось обучение полносвязной сети из четырех слоев без предобучения и с послойным автоэнкодерным предобучением на пяти разбиениях. На датасете HCV без предобучения получены ассурасу 0.254 ± 0.021 и масро F1 0.251 ± 0.021 , с предобучением ассурасу 0.258 ± 0.014 и масро F1 0.255 ± 0.013 . Для четырех почти равных классов случайное угадывание дает около 0.25,

то есть обе модели работают на этом уровне, а прирост в 0.004 по accuracy и macro F1 намного меньше разброса между запусками и значимым не является. Отчет по классам подтверждает это: F1 по классам лежит в диапазоне от 0.21 до 0.31, а recall от 0.20 до 0.36. Скорее всего, причина в самих данных, так как признаки слабо связаны со стадией фиброза. На датасете Mushroom обе модели показали accuracy, macro F1 и weighted F1, равные 1.000, со стандартным отклонением 0 на всех разбиениях (1694 тестовых объекта, ошибок нет), потому что после one hot кодирования классы разделяются практически идеально. Итого предобучение не улучшило итоговое качество ни на одном из датасетов, а его возможная польза при большом числе признаков и малом объеме размеченных данных в этих экспериментах не проявилась.

Вывод: научилась применять автоэнкодеры для осуществления визуализации данных и их анализа.